

Complete Guide to Async/Await in JavaScript

Table of Contents

1. Introduction to Async/Await
 2. Understanding Async Functions
 3. Understanding Await Keyword
 4. Practical Project: GitHub Users Display
 5. Error Handling with Try-Catch
 6. Converting Promise Chains to Async/Await
 7. Promise.all() for Parallel Execution
 8. Best Practices
-

1. Introduction to Async/Await

Async/Await is a **modern and cleaner way** to handle asynchronous operations in JavaScript. It makes asynchronous code look and behave more like synchronous code, making it easier to read and maintain.

Why Async/Await?

Previous approaches were complex:

- **Callback Hell:** Nested callbacks made code difficult to read
- **Promise Chains:** `.then()` and `.catch()` chains were better but still complex
- **Async/Await:** The cleanest and most readable approach

Important: Async/Await is considered the "heart of JavaScript" for both frontend and backend development.

2. Understanding Async Functions

Basic Async Function

```
async function greet() {
```

```
    return "Rohit";
  }

const response = greet();
console.log(response); // Output: Promise { 'Rohit' }
```

Key Rule #1: Async functions ALWAYS return a Promise

When you add the `async` keyword before a function:

- The function automatically returns a Promise
- If you return a simple value, it wraps that value in a Promise
- If you return a Promise, it returns that Promise as-is

Two Ways to Return Promises

Method 1: Direct value (auto-wrapped)

```
async function greet() {
  return "Rohit";
}
```

Method 2: Explicit Promise

```
async function greet() {
  return new Promise((resolve, reject) => {
    resolve("Rohit");
  });
}
```

Both methods are equivalent!

Consuming Async Function Results

Using `.then()` (traditional way)

```
async function greet() {
  return "Rohit";
}
```

```
const response = greet();
response.then(data => {
```

```
    console.log(data); // Output: Rohit
  });
```

Handling Rejections

```
async function greet() {
  return new Promise((resolve, reject) => {
    reject("Error occurred");
  });
}

const response = greet();
response
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error); // Handles rejections
  });
```

3. Understanding the Await Keyword

The Problem Without Await

```
// This DOESN'T work correctly
const response = fetch('https://api.github.com/users');
const data = response.json(); // ERROR! response is still a pending Promise
console.log(data);
```

Why it fails:

- JavaScript doesn't wait for asynchronous operations
- It immediately moves to the next line
- `response` is a Promise in pending state
- Trying to call `.json()` on a pending Promise causes an error

The Solution: Using Await

```
const response = await fetch('https://api.github.com/users');
const data = await response.json();
console.log(data); // Works correctly!
```

What **await** does:

- **Pauses execution** of the async function
- Waits for the Promise to resolve
- Then moves to the next line
- Makes asynchronous code appear synchronous

Key Rule #2: Await can only be used inside async functions

```
// CORRECT: await inside async function
async function getUsers() {
  const response = await fetch('https://api.github.com/users');
  const data = await response.json();
  console.log(data);
}

getUsers();
```

4. Why Wrap Await in Async Functions?

The Problem with Top-Level Await

```
// DON'T DO THIS (before 2022)
const response = await fetch('https://api.github.com/users');
const data = await response.json();
console.log("Hello, how are you?");
```

Problems:

1. **Freezes the entire program** at the await line
2. Nothing below executes until the Promise resolves
3. Even independent code (like "Hello, how are you?") is blocked
4. Your entire web page becomes unresponsive

The Correct Approach

```
async function getUsers() {
  const response = await fetch('https://api.github.com/users');
  const data = await response.json();
  console.log(data);
}
```

```
// Other independent code
console.log("Hello, how are you?"); // Executes immediately
```

```
// Call the async function
getUsers(); // Executes independently without blocking
```

Execution Flow:

1. "Hello, how are you?" prints immediately
2. `getUsers()` starts executing
3. When it hits `await`, it releases control
4. JavaScript continues with other tasks
5. When data arrives, it resumes `getUsers()`

Note: Since 2022, top-level `await` is supported in modern JavaScript, but wrapping in `async` functions is still recommended for better control.

5. Practical Project: GitHub Users Display

HTML Structure (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>GitHub Users</title>
  <link rel="stylesheet" href="./style.css">
</head>
<body>
  <h1>GitHub Users</h1>
  <div id="first"></div>
  <script src="./script.js"></script>
</body>
</html>
```

CSS Styling (style.css)

```
* {
  box-sizing: border-box;
```

```
padding: 0;  
margin: 0;  
}
```

```
body {  
  width: 100%;  
  height: 100vh;  
  background-color: black;  
}
```

```
h1 {  
  color: white;  
  font-size: 40px;  
  text-align: center;  
  margin-top: 20px;  
  margin-bottom: 20px;  
}
```

```
#first {  
  display: flex;  
  justify-content: center;  
  width: 80%;  
  margin: auto;  
  gap: 30px;  
  flex-wrap: wrap;  
}
```

```
.user {  
  width: 200px;  
  text-align: center;  
  padding: 5px;  
}
```

```
.user img {  
  width: 100%;  
  height: 200px;  
  object-fit: cover;  
  border: 2px solid white;  
  border-radius: 5px;  
}
```

```
.user h2 {  
  font-size: 20px;  
  color: white;
```

```

margin-top: 10px;
margin-bottom: 10px;
}

.user a {
  text-decoration: none;
  color: orange;
  padding-bottom: 10px;
}

.user: hover {
  transform: scale(1.2);
  transition: transform 1s ease-in;
}

```

JavaScript Implementation (script.js)

```

async function getUsers() {
  try {
    // Fetch users from GitHub API
    const response = await fetch('https://api.github.com/users');

    // Check if response is okay
    if (!response.ok) {
      throw new Error('Data is not present');
    }

    // Convert response to JSON
    const data = await response.json();

    // Get parent container
    const parent = document.getElementById('first');

    // Loop through each user
    for (let user of data) {
      // Create main container div
      const element = document.createElement('div');
      element.classList.add('user');

      // Create and configure image element
      const image = document.createElement('img');
      image.src = user.avatar_url;

      // Create username heading

```

```

    const username = document.createElement('h2');
    username.textContent = user.login;

    // Create profile link
    const anchor = document.createElement('a');
    anchor.href = user.html_url;
    anchor.textContent = 'Visit Profile';

    // Append all elements to container
    element.append(image, username, anchor);

    // Append container to parent
    parent.append(element);
  }

} catch (error) {
  console.log(error);
}

}

// Call the function
getUsers();

```

API Response Structure

The GitHub API returns an array of user objects:

```

[
  {
    "login": "mojombo",
    "avatar_url": "https://avatars.githubusercontent.com/u/1?v=4",
    "html_url": "https://github.com/mojombo",
    // ... other properties
  },
  // ... 29 more users
]

```

Key Properties Used:

- `login`: Username
- `avatar_url`: Profile picture URL
- `html_url`: GitHub profile link

6. Error Handling with Try-Catch

Why Error Handling is Important

Potential errors:

1. Network is unreachable
2. Server is down
3. DNS issues
4. API limit exceeded
5. Invalid response format

Complete Error Handling Example

```
async function getUsers() {  
  try {  
    // Step 1: Fetch data  
    const response = await fetch('https://api.github.com/users');  
  
    // Step 2: Check if response is valid  
    if (!response.ok) {  
      throw new Error('Data is not present');  
    }  
  
    // Step 3: Parse JSON  
    const data = await response.json();  
  
    // Step 4: Process data  
    console.log(data);  
  
  } catch (error) {  
    // Handles ALL errors from try block  
    console.log(error);  
  }  
}
```

Understanding Response.ok

```
// What is response.ok?  
const response = await fetch('https://api.github.com/users');  
  
if (!response.ok) {
```

```

// response.ok is false when:
// - Status code is 400+ (client errors)
// - Status code is 500+ (server errors)
// Example: 404 Not Found, 500 Internal Server Error

throw new Error('Data is not present');
}

// response.ok is true when:
// - Status code is 200-299 (success)

```

Best Practice Pattern

```

async function fetchData() {
  try {
    // All async operations in try block
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error('Failed to fetch');
    }

    const data = await response.json();
    return data;
  } catch (error) {
    // Handle any error that occurs
    console.error('Error:', error);
    throw error; // Re-throw if needed
  }
}

```

7. Converting Promise Chains to Async/Await

Old Way: Promise Chaining (Complex)

```

function placeOrder(orderDetail) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      console.log('Payment is in progress...');
      resolve(orderDetail);
    }, 2000);
  });
}

```

```
});  
}
```

```
function preparingOrder(orderDetail) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log('Order is being prepared...');  
      resolve(orderDetail);  
    }, 1000);  
  });  
}
```

```
function pickupOrder(orderDetail) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log('Order picked up...');  
      resolve(orderDetail);  
    }, 1000);  
  });  
}
```

```
function deliverOrder(orderDetail) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log('Order delivered!');  
      resolve(orderDetail);  
    }, 1000);  
  });  
}
```

// COMPLEX Promise Chain

```
const orderDetail = { item: 'Pizza', price: 500 };
```

```
placeOrder(orderDetail)  
  .then((order) => {  
    return preparingOrder(order);  
  })  
  .then((order) => {  
    return pickupOrder(order);  
  })  
  .then((order) => {  
    return deliverOrder(order);  
  })  
  .then((order) => {
```

```
    console.log('Complete:', order);
  })
  .catch((error) => {
    console.log(error);
  });
```

New Way: Async/Await (Clean & Simple)

```
async function ordering() {
  try {
    const orderDetail = { item: 'Pizza', price: 500 };

    // Step 1: Place order
    const response1 = await placeOrder(orderDetail);

    // Step 2: Prepare order
    const response2 = await preparingOrder(response1);

    // Step 3: Pickup order
    const response3 = await pickupOrder(response2);

    // Step 4: Deliver order
    const response4 = await deliverOrder(response3);

    console.log('Complete:', response4);

  } catch (error) {
    console.log(error);
  }
}

ordering();
```

Benefits of Async/Await

Readability:

- Code reads like synchronous operations
- Clear sequential flow
- Easy to understand

Maintainability:

- Easy to add steps
- Easy to remove steps
- Simple error handling

Debugging:

- Easier to set breakpoints
 - Better stack traces
 - Clearer error messages
-

8. Promise.all() for Parallel Execution

The Problem: Sequential Execution

```
async function userDetails() {  
  // These run ONE BY ONE (sequential)  
  const comments = await fetch('/api/comments');  
  const photos = await fetch('/api/photos');  
  const chats = await fetch('/api/chats');  
  
  // SLOW! Each waits for the previous one to complete  
}
```

Problems:

- Comments fetch: 2 seconds
- Photos fetch: 2 seconds
- Chats fetch: 2 seconds
- **Total time: 6 seconds** (sequential)

The Solution: Promise.all()

```
async function userDetails() {  
  // All three requests start AT THE SAME TIME  
  const [comments, photos, chats] = await Promise.all([  
    fetch('/api/comments'),  
    fetch('/api/photos'),  
    fetch('/api/chats')  
  ]);  
  
  // FAST! All requests run in parallel  
  // Total time: ~2 seconds (the slowest request)
```

```
}
```

Benefits:

- All requests start simultaneously
- Wait only for the slowest one
- Massive performance improvement

Complete Example

```
async function fetchUserData() {
  try {
    console.log('Fetching data...');

    // Start all requests in parallel
    const [commentsRes, photosRes, chatsRes] = await Promise.all([
      fetch('https://api.example.com/comments'),
      fetch('https://api.example.com/photos'),
      fetch('https://api.example.com/chats')
    ]);

    // Parse all responses in parallel
    const [comments, photos, chats] = await Promise.all([
      commentsRes.json(),
      photosRes.json(),
      chatsRes.json()
    ]);

    console.log('Comments:', comments);
    console.log('Photos:', photos);
    console.log('Chats:', chats);

  } catch (error) {
    console.log('Error:', error);
  }
}
```

Real-World Use Case: Multiple Database Connections

```
async function connectDatabases() {
  try {
    // Connect to multiple databases in parallel
    const [mongoConnection, postgresConnection, redisConnection] =
      await Promise.all([
```

```
        connectToMongoDB(),
        connectToPostgres(),
        connectToRedis()
    ]);

    console.log('All databases connected!');

} catch (error) {
    console.log('Database connection failed:', error);
}
}
```

9. Best Practices Summary

✅ DO's

1. Always use async functions with await

```
async function fetchData() {
    const data = await fetch(url);
    return data;
}
```

2. Always use try-catch for error handling

```
async function fetchData() {
    try {
        const data = await fetch(url);
        return data;
    } catch (error) {
        console.error(error);
    }
}
```

3. Check response.ok before processing

```
const response = await fetch(url);
if (!response.ok) {
    throw new Error('Request failed');
}
```

4. Use Promise.all() for parallel operations

```
const [data1, data2] = await Promise.all([
  fetch(url1),
  fetch(url2)
]);
```

5. Add appropriate waiting messages

```
console.log("Loading...");
const data = await fetch(url);
console.log("Loaded!");
```

DON'Ts

1. Don't use await outside async functions (in older JavaScript)

```
// BAD (before 2022)
const data = await fetch(url); // Error!
```

2. Don't forget error handling

```
// BAD - No error handling
async function fetchData() {
  const data = await fetch(url);
}
```

3. Don't use sequential await for independent operations

```
// BAD - Unnecessary waiting
const data1 = await fetch(url1);
const data2 = await fetch(url2); // Should use Promise.all()
```

4. Don't ignore failed responses

```
// BAD - No response check
const data = await response.json(); // Might fail!
```

10. Complete Cheat Sheet

Basic Syntax

```
// Async function declaration
async function functionName() {
  const result = await asyncOperation();
  return result;
}

// Async arrow function
const functionName = async () => {
  const result = await asyncOperation();
  return result;
};

// Async method in class
class MyClass {
  async myMethod() {
    const result = await asyncOperation();
    return result;
  }
}
```

Error Handling Patterns

```
// Pattern 1: Try-Catch
async function pattern1() {
  try {
    const data = await fetch(url);
    return data;
  } catch (error) {
    console.error(error);
  }
}

// Pattern 2: Catching specific errors
async function pattern2() {
  try {
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
  }
}
```

```

        return await response.json();
    } catch (error) {
        if (error.message.includes('HTTP error')) {
            console.log('Server error');
        } else {
            console.log('Network error');
        }
    }
}

```

Common Use Cases

// 1. Fetching data

```

async function getData() {
    try {
        const response = await fetch('https://api.example.com/data');
        if (!response.ok) throw new Error('Failed');
        const data = await response.json();
        return data;
    } catch (error) {
        console.error(error);
    }
}

```

// 2. Sequential operations

```

async function sequential() {
    const step1 = await operation1();
    const step2 = await operation2(step1);
    const step3 = await operation3(step2);
    return step3;
}

```

// 3. Parallel operations

```

async function parallel() {
    const [result1, result2, result3] = await Promise.all([
        operation1(),
        operation2(),
        operation3()
    ]);
    return { result1, result2, result3 };
}

```

// 4. Race condition (first one wins)

```
async function race() {  
  const result = await Promise.race([  
    fetch('https://api1.com/data'),  
    fetch('https://api2.com/data')  
  ]);  
  return result;  
}
```

Summary

Async/Await revolutionizes asynchronous JavaScript by:

1. Making async code look synchronous
2. Improving readability dramatically
3. Simplifying error handling
4. Enabling better debugging
5. Supporting parallel execution with `Promise.all()`

Remember:

- Async functions always return Promises
- Await pauses execution until Promise resolves
- Always wrap await in async functions
- Use try-catch for error handling
- Use `Promise.all()` for parallel operations

This is the modern standard for handling asynchronous operations in JavaScript, and it's essential for both frontend and backend development!